HPO-Adapt: A Diagnostic-Driven Toolkit for Adaptive Hyperparameter-Optimization Setup Control

York Wang*

Abstract

The performance of hyperparameter optimization (HPO) is largely determined before the first trial runs: the initial *setup*—search space, optimizer, budget—fixes what can be found at all. Analysis frameworks such as CAVE and DeepCAVE expose what went wrong *afterwards* but leave the repair to the user, while LLM-agent approaches tend to replace classical HPO rather than repair its setup. We present HPO-Adapt, a Python toolkit that closes this loop automatically: a deterministic *Diagnostic Engine* computes a structured state from the trial trajectory, a tabular Q-learning *Controller* selects one of seven high-level interventions, a *Revised Setup Agent* executes it as a validated new setup, and an *Agent Memory* retains experiences across episodes. Every agent has a deterministic offline backend, so the full loop runs without any API key and is exactly reproducible. On two benchmark tasks whose initial setup is misspecified, guided adaptation reaches substantially better optima than equally-budgeted static HPO (1.000 vs. 0.755; 0.498 vs. 0.896, approaching the global minimum), while on a real RandomForest task it matches strong static baselines within noise; the controller keeps improving across episodes, and the observe–decide layer costs 1.4% of wall-clock on real training workloads.

1 Introduction

Hyperparameter optimization automates the search for good model configurations, but it silently inherits every flaw of its initial setup: a search space that is too narrow truncates the optimum, an unsuited optimizer or budget wastes the search [1]. Practitioners usually discover such flaws only after the budget is gone, by inspecting the HPO trajectory with tools such as CAVE [2] or DeepCAVE [3]. These frameworks make HPO transparent, but *repairing* the setup remains a manual, experience-driven step. A second line of work assigns the setup to LLM agents [4, 5]; recent evidence indicates that LLMs do not outperform classical HPO algorithms as searchers [6], and such agents remain bound by the bounds and interfaces they are given [7]. Neither line answers the question that matters during a running search: *given the trajectory so far, which concrete change to the current setup is most promising right now?*

HPO-Adapt answers it with a closed loop and a strict separation of concerns. **Observe**: a deterministic *Diagnostic Engine* reduces the cumulative trial trajectory to a structured `DiagnosticState` (importance, boundary concentration, good regions, diversity, coverage, budget), with explicit `VALID`/`INSUFFICIENT_DATA` statuses instead of fabricated numbers. **Decide**: a tabular Q-learning *Controller* [8] maps that state to one of seven high-level interventions. **Execute**: a *Revised Setup Agent* turns the intervention into a new setup that must pass a tool validator before any HPO run; an *Agent Memory* (SQLite) stores completed adaptation experiences. Both agents also ship deterministic heuristic backends, so the system runs offline, is seeded end-to-end, and every number in this paper regenerates with one command. Contributions: **(i)** a modular open-source toolkit that turns CAVE-style trajectory diagnosis into *acted-on* setup adaptation rather than after-the-fact plots; **(ii)** a design that keeps metric computation free of LLMs and the agent role strictly at validated setup construction; **(iii)** 126 unit tests and one-command reproducible experiments; and **(iv)** an empirical study showing that, on misspecified setups, guided adaptation clearly beats equally-budgeted static HPO while unguided intervention does not, that the controller improves across episodes, and that the observe–decide layer costs 1.4% of wall-clock on real training workloads.

2 Related Work

HPO and its setup problem. Classical HPO samples configurations from a fixed search space [1]; Auto-WEKA already framed the joint choice of model and hyperparameters as a search problem [9]. Practical HPO relies on

*Course project; the toolkit, experiments and this paper were produced as part of a graduate AutoML course project. Code: `https://github.com/yorkwang/hpo-adapt` (replace with the final repository URL).

model-based tools such as SMAC3 [10] and Optuna [11]; HPO-Adapt builds on Optuna and targets the situation these tools leave to the user: the setup was chosen once and may be wrong.

Trajectory analysis. CAVE [2] and DeepCAVE [3] assess finished HPO runs (importance, configuration footprints): they inform humans but do not feed back into the running search. HPO-Adapt reuses their core indicators—fANOVA-style importance and local-perturbation importance—but recomputes them *during* the search, on the cumulative trajectory, as one structured state that a controller can act on.

LLM agents for AutoML. AgentHPO [4] and AutoML-Agent [5] delegate setup and even search to LLM agents; such agents remain bound by externally imposed bounds and interfaces [7], and controlled studies find classical HPO still stronger as the searcher [6]. HPO-Adapt keeps classical HPO as the searcher and assigns agents only the setup, with every output schema-validated—a design that also makes the system runnable fully offline through deterministic heuristic backends.

3 The HPO-Adapt Toolkit

3.1 Closed loop and data objects

[FIGURE:] Fig. 1: closed loop Task → Initial Setup → S_0 → HPO (Optuna) → TrialLogs → Diagnostic Engine → RL Controller → Revised Setup → S_{t+1}, with the Tool Validator gate and SQLite memory. Full spec: paper/FIGURES.md Fig. 1.

The system is organized around six pydantic schemas: `TaskSpec` (objective, constraints, and the *allowed* hyperparameter metadata), `Setup` (search space, optimizer, budget, evaluation), `TrialLog`, `DiagnosticState`, `RLAction` ($A_t = (\text{ActionType}, \text{Target}, \text{Parameters})$), and `Experience`. Every setup must pass `validate_setup` before the HPO runner accepts it: the validator rejects invented hyperparameters, out-of-domain ranges, type or direction mismatches, unknown optimizers, and budget violations.

3.2 Diagnostic Engine (Observe)

The engine is pure, deterministic Python; no LLM sees raw logs. It analyzes the *cumulative* trajectory of the current episode and returns a state with three sections. *Optimization status*: best objective $B_t = \max_{i \le t} \tilde{y}_i$, recent improvement over an analysis window w, $RI_t = (B_t - B_{t-w})/(|B_{t-w}| + \epsilon)$, stagnation $\mathbb{1}(RI_t < \epsilon_{\text{stag}})$, and marginal gain per trial/second, computed on internally maximized values $\tilde{y}_i = y_i$ (maximize) or $-y_i$ (minimize). *Search-space status*: per hyperparameter, (i) global importance from an fANOVA-style variance decomposition over a random-forest surrogate plus local importance from leave-one-out prediction perturbation at the best configurations [12]; (ii) boundary concentration of all trials and of the top-q% trials (per upper/lower edge)—the key truncation signal; (iii) the *good region* (mean and $[10\%, 90\%]$ quantile interval, width ratio vs. the current range; category frequencies for categoricals); (iv) configuration diversity (mixed-type pairwise distance of the recent window vs. the full trajectory) and (v) log-space-aware per-dimension coverage. *Budget status*: consumed/remaining share of the global budget. Every metric carries a status: below the minimum trial counts (5/20/30 for basic/stagnation/importance metrics) the engine reports `INSUFFICIENT_DATA` instead of a value, and hyperparameter-targeted actions stay unavailable until importance becomes `VALID`—the system never fabricates evidence.

3.3 RL Controller (Decide)

The global state discretizes stagnation, recent improvement, diversity, coverage, budget remaining, and marginal gain into low/medium/high; up to two candidate hyperparameters (ranked by $\alpha\,\text{Imp}_j + \beta\,BC_j^{\text{good}} + \gamma\,(1 - \text{WidthRatio}_j)$, $\alpha = .5, \beta = .3, \gamma = .2$) add per-candidate context. Actions are the seven interventions in Table 1. Selection is ϵ-greedy with decay; ties among equally-good actions are broken by a rule engine that encodes seven diagnostic patterns (e.g., *stagnation + high good-boundary concentration + high importance* ⇒ EXPAND; *low importance for multiple rounds* ⇒ FREEZE), so the rules act as an interpretable prior while Q-values refine it from experience. STOP is evidence-gated: selectable only under observed stagnation with at least half the budget consumed, preventing unguided premature termination. The reward is $r = \text{clip}(\Delta - \lambda C - r_{\text{invalid}}, \pm r_{\max})$ with $\Delta = (B_{t+1} - B_t)/(|B_t| + \epsilon)$ the relative improvement caused by the intervention, C the budget share it consumed, and a fixed penalty for structurally invalid actions (which never consume budget and are retried within the round).

Table 1: The seven setup interventions (A_t = action, target, parameters).

Action (target)	Semantics
ExpandSearchSpace (numeric hp)	widen range toward the boundary indicated by good trials
ShiftSearchRegion (numeric hp)	translate range toward the observed good region
ShrinkSearchSpace (numeric hp)	focus range on the good region (with margin)
FreezeHyperparameter (any hp)	fix a persistently low-importance parameter
IncreaseExploration (optimizer)	raise exploration / switch optimizer
IncreaseBudget (budget)	enlarge the next phase's trial budget (capped)
Stop (—)	terminate the episode (evidence-gated)

3.4 Setup Agents (Execute) and Memory

The *Initial Setup Agent* constructs S_0 from task metadata, constraints, and memory context; the *Revised Setup Agent* applies the selected intervention *conservatively*: only the targeted component changes, expansions are clipped to the task's metadata domain, shifts preserve width, shrinks never fall below a minimum relative width, and freezes prefer the task default. Both agents have a deterministic heuristic backend (default; used for all experiments) and an LLM backend (any OpenAI-compatible endpoint) with identical interfaces; every output passes the same validator, so nothing violating the schema or task constraints is ever executed. Phases are *warm-started*: each new Optuna study is seeded with all past completed trials compatible with the current search space, so a revised setup inherits every evaluation instead of restarting the optimizer's random startup; only newly evaluated trials consume budget, keeping static and adaptive runs comparable.

The *Agent Memory* is a SQLite store with two tables: `experiences` (one row per completed adaptation step: task signature, setup before/after, diagnostic state, diagnosis, action, outcome, reward) and `semantic_rules`, which aggregates outcomes into rule-level success rates. By default only successful interventions and instructive failures are persisted. Memory answers "what happened in similar situations before?" while the Q-table answers "what pays off in this state?"—complementary roles.

3.5 Using the toolkit

HPO-Adapt installs with `pip install -e .` and offers a CLI (`hpo-adapt -task synthetic_mlp -max-rounds 4 -seed 3`), a Python API (`AdaptationPipeline(task, PipelineConfig()).run()`), and plug-in points for custom objectives: register the objective function, describe the task's hyperparameter metadata in a `TaskSpec`, and run the pipeline. The shipped tasks cover a synthetic MLP-style benchmark whose optimum lies outside a default-derived learning-rate range, the Branin function (minimize), and a real RandomForest/3-fold-CV task on sklearn digits.

4 Evaluation

4.1 Setup

We compare, under an *identical* total budget of 120 evaluations per run and 10 seeds each: static baselines that spend the whole budget in one phase (`static-random`, `static-tpe`) and four adaptive variants (phases of 40 trials, at most three phases): `adapt-random-action` (unguided control), `adapt-rule-only` (always the rule engine's recommendation), `adapt-rl-default` (spec-default ϵ-greedy $1.0 \to 0.05$), and `adapt-rl-guided` ($\epsilon_0 = 0.2$). A second experiment shares Q-table and memory across 20 consecutive episodes per (task, seed). All runs use the deterministic heuristic agent backends; every number below regenerates with `run_all.py` + `make_figures.py`.

4.2 Main results

Table 2: Final best objective (mean±std, 10 seeds, raw space; Branin minimized; best per task in **bold**).

	synthetic_mlp (↑)	Branin (↓)	rf_digits (↑)
static-random	0.657±0.039	1.759±0.426	0.9766±0.0008
static-tpe	0.755±0.003	0.896±0.010	**0.9772±0.0004**
adapt-random-action	0.873±0.134	0.776±0.258	0.9767±0.0008
adapt-rule-only	**1.000±0.000**	**0.498±0.198**	0.9771±0.0005
adapt-rl-default	0.873±0.134	0.776±0.258	0.9767±0.0008
adapt-rl-guided	**1.000±0.000**	0.596±0.251	0.9771±0.0005

[FIGURE:] Fig. 2: mean best-so-far vs. trials, 6 methods × 3 tasks; dashed = static. Data: `figs/anytime.csv` (FIGURES.md Fig. 2).

On the synthetic task the picture is unambiguous: guided adaptation reaches the achievable optimum (1.000±0.000, i.e., every seed recovers learning rate ≈ 0.42–0.45), while static TPE tops out at 0.755±0.003—a 32% relative gap. On Branin, `adapt-rule-only` reaches 0.498±0.198 against static TPE's 0.896±0.010, approaching the global minimum 0.398. On the real task all methods land within 0.0006 of each other (0.9766–0.9772): the metadata-driven initial setup is already near the CV-accuracy plateau, so adaptation neither helps nor hurts—the toolkit's value there is observability.

4.3 Ablations: guidance matters

Comparing the three adaptation policies isolates the contribution of guidance. On synthetic_mlp, `adapt-rule-only` and `adapt-rl-guided` achieve 1.000±0.000 (worst seed = best seed), whereas `adapt-random-action` manages only 0.873±0.134—a six-fold larger spread. On Branin the ordering is the same: 0.498±0.198 and 0.596±0.251 for the guided variants against 0.776±0.258 for the random control. Intervening *without* evidence buys little and costs variance. Under spec-default exploration, `adapt-rl-default` behaves exactly like the random control within a single episode ($\epsilon \approx 1$ over 2–3 acting rounds), so the value of its Q-table is measured across episodes instead (below).

4.4 Learning across episodes

[FIGURE:] Fig. 3: per-episode best over 20 episodes, shared Q-table/memory vs. random control and static-TPE, two panels. Data: `figs/episodes_summary.csv` (FIGURES.md Fig. 3).

With the Q-table and memory shared across 20 consecutive episodes, `adapt-rl` improves as experience accumulates: on synthetic_mlp the mean per-episode best rises from 0.892 over the first five episodes to 0.949 over the last five (peak 0.975 at episode 5), while the unguided control stays at 0.867→0.855 and static TPE is flat at ≈0.75. On Branin (minimize) `adapt-rl` improves from 0.734 to 0.524 (0.469 already at episode 5) while the unguided control drifts from 0.768 to 0.819; static TPE stays at ≈0.90. The Q-table stays small (3 states on average), so learning remains tractable.

4.5 Case study: detecting and repairing truncation

[FIGURE:] Fig. 4 (full width): (a) best-so-far with round actions; (b) learning-rate interval per round (log scale) with GT 0.45 and sampled points; (c) reward per round. Data: `figs/case_study.json` (FIGURES.md Fig. 4).

Fig. 4 traces `adapt-rule-only` (seed 0) on synthetic_mlp. After round 0 (40 trials), cumulative importance marks learning_rate as dominant (0.876) and 75% of the top-10% trials sit at the *upper* boundary of the initial range $[10^{-3}, 10^{-1}]$: ShiftSearchRegion moves the interval to $[9.1\times10^{-3}, 9.1\times10^{-1}]$, preserving its (log-)width. The warm-started TPE phase samples the newly opened region immediately: the best jumps from 0.743 to 1.000, recovering learning rate 0.420 (ground truth 0.45), and a second Shift recenters the interval on the good region. Round 2 spends the remaining budget and closes the episode without a further action.

4.6 Cost and reproducibility

On rf_digits, where each trial actually trains models, the observe–decide–execute layer (diagnostics, rule engine, Q-updates, memory, agent execution) costs 1.4% of wall-clock time on average (1.7% worst case over the 60 adaptive runs); the remainder is model training. On analytic objectives trials cost microseconds, so that layer trivially dominates. The MIT-licensed toolkit ships 126 unit tests covering every module; with fixed seeds, every table above regenerates from one command.

5 Conclusion

HPO-Adapt closes the loop from CAVE-style trajectory diagnosis to schema-validated setup revision: guided adaptation substantially improves results on misspecified setups at 1.4% overhead, and the controller improves across episodes. Future work: LLM backends in the loop and embedding-based memory retrieval.

References

[1] Matthias Feurer and Frank Hutter. Hyperparameter optimization. In *Automated Machine Learning: Methods, Systems, Challenges*, pages 3–33. Springer, 2019.

[2] Katharina Eggensperger, Philipp Muller, Neeratyoy Mallik, Marius Lindauer, and Frank Hutter. CAVE: Configuration assessment, visualization and evaluation. *arXiv preprint arXiv:1907.02101*, 2019.

[3] Sarah Segel, Difan Deng, Neeratyoy Mallik, Frank Hutter, and Marius Lindauer. DeepCAVE: A visualization and analysis tool for automated machine learning. *Journal of Machine Learning Research*, 26(64):1–27, 2025. Machine Learning Open Source Software (MLOSS) track.

[4] Shun Liu et al. AgentHPO: Large language model agent for hyper-parameter optimization. *arXiv preprint arXiv:2502.02089*, 2025. Citation details to be verified against the final published version.

[5] Parinthorn Premchaiswadi, Anuchit Premchaiswadi, et al. AutoML-Agent: A multi-agent LLM framework for full-pipeline AutoML. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. Citation details to be verified against the final published version.

[6] Fabio Ferreira, Lennart Wobbe, Arjun Krishnakumar, et al. Can LLMs beat classical hyperparameter optimization algorithms? A study on autoresearch. *arXiv preprint arXiv:2603.24647*, 2026. Citation details to be verified against the final published version.

[7] Tanja Tornede, Alexander Tornede, Marcel Wever, and Eyke Hüllermeier. AutoML in the age of large language models: Current challenges, future opportunities and risks. *Transactions on Machine Learning Research*, 2024.

[8] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[9] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 847–855, 2013.

[10] Marius Lindauer, Katharina Eggensperger, Matthias Feurer, André Biedenkapp, Difan Deng, Carolin Benjamins, Tim Ruhkopf, René Sass, and Frank Hutter. SMAC3: A versatile bayesian optimization package for hyperparameter optimization. *Journal of Machine Learning Research*, 23(54):1–9, 2022.

[11] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2623–2631, 2019.

[12] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. An efficient approach for assessing hyperparameter importance. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pages 754–762, 2014.